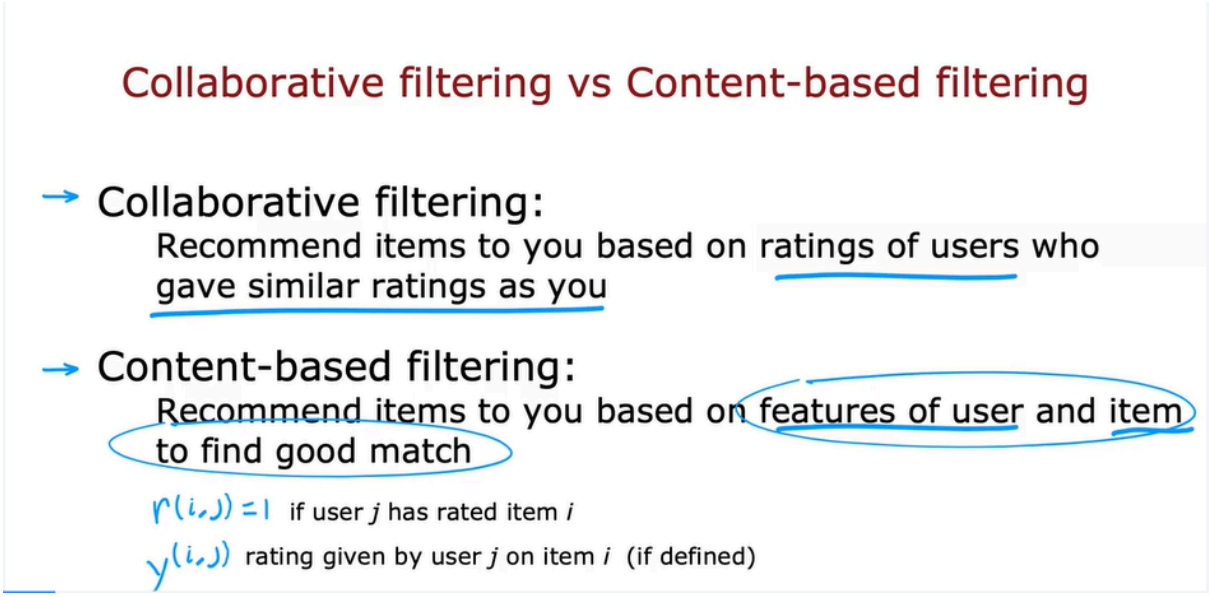
Content-based filtering

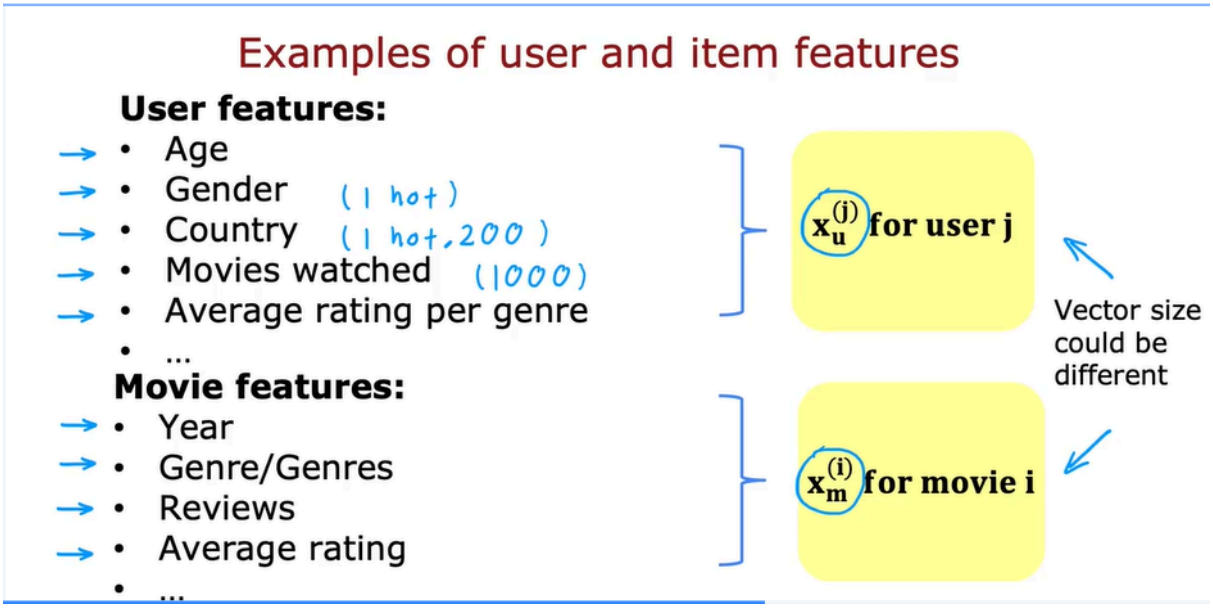
Collaborative filtering vs Content-based filtering

In this video, we'll start to develop a second type of recommender system called a content-based filtering algorithm. To get started, let's compare and contrast the collaborative filtering approach that we'll be looking at so far with this new content-based filtering approach.



- With collaborative filtering, the general approach is that we would recommend items to you based on ratings of users who gave similar ratings as you. We have some number of users give some ratings for some items, and the algorithm figures out how to use that to recommend new items to you. In contrast, content-based filtering takes a different approach to deciding what to recommend to you.
- A content-based filtering algorithm will recommend items to you based on the features of users and features of the items to find a good match. In other words, it requires having some features of each user, as well as some features of each item and it uses those features to try to decide which items and users might be a good match for each other.

- With a content-based filtering algorithm, you still have data where users have rated some items. Well, content-based filtering will continue to use r(i, j) to denote whether or not user j has rated item i and will continue to use y(i, j) to denote the rating that user j is given item i if it's defined. But the key to content-based filtering is that we will be able to make good use of features of the user and of the items to find better matches than potentially a pure collaborative filtering approach might be able to.



Let's take a look at how this works.

In the case of movie recommendations, here are some examples of features. You may know the age of the user, or you may have the gender of the user. This could be a one-hot feature similar to what you saw when we were talking about decision trees where you could have a one-hot feature with the values based on whether the user's self-identified gender is male or female or unknown, and you may know the country of the user. If there are about 200 countries in the world then also be a one-hot feature with about 200 possible values. You can also look at past behaviors of the user to construct this feature vector.

- For example, if you look at the top thousand movies in your catalog, you might construct a thousand features that tells you of the thousand most popular movies in the world which of these has the user watch. In fact, you can also take ratings the user might have already given in order to construct new features. It turns out that if you have a set of movies and if you know what genre each movie is in, then the average rating per genre that the user

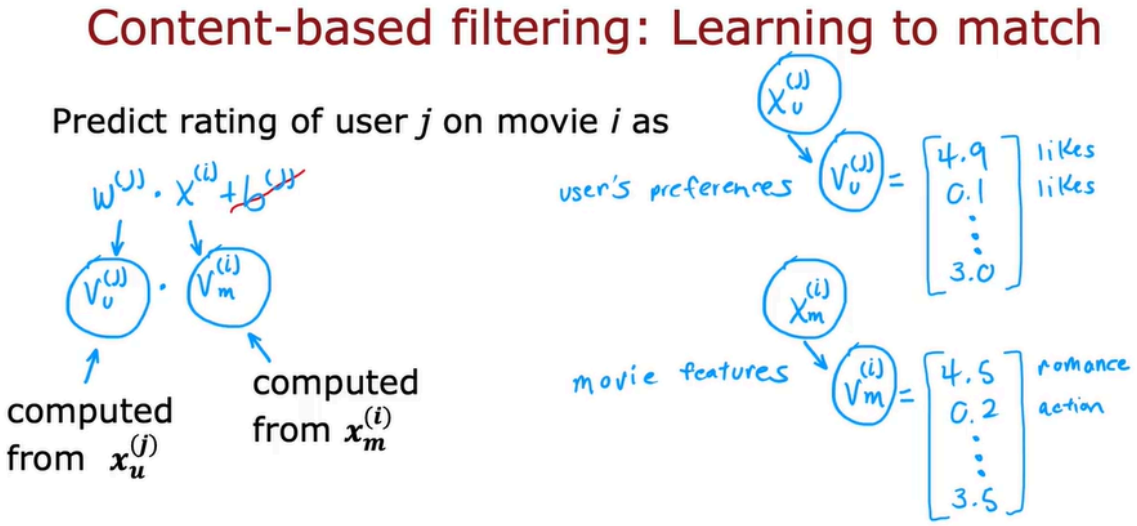
has given. Of all the romance movies that the user has rated, what was the average rating? Of all the action movies that the user has rated, what was the average rating? And so on for all the other genres. This too can be a powerful feature to describe the user. One interesting thing about this feature is that it actually depends on the ratings that the user had given. But there's nothing wrong with that.

Constructing a feature vector that depends on the user's ratings is a completely fine way to develop a feature vector to describe that user. With features like these you can then come up with a feature vector $x_u^{(j)}$ for user j. Similarly, you can also come up with a set of features for each movie of each item, such as what was the year of the movie? What's the genre or genres of the movie of known? If there are critic reviews of the movie, you can construct one or multiple features to capture something about what the critics are saying about the movie.

Or once again, you can actually take user ratings of the movie to construct a feature of, say, the average rating of this movie. This feature again depends on the ratings that users are given but again, does nothing wrong with that. You can construct a feature for a given movie that depends on the ratings that movie had received, such as the average rating of the movie. Or if you wish, you can also have average rating per country or average rating per user demographic as they want to construct other types of features of the movies as well. With this, for each movie, you can then construct a feature vector, which I'm going to denote $x_m^{(i)}$ m stands for movie.

Given features like this, the task is to try to figure out whether a given movie i is going to be good match for user j. Notice that the user features and movie features can be very different in size. For example, maybe the user features could be 1500 numbers and the movie features could be just 50 numbers. That’s okay too

Content-based filtering: learning to match



In content-based filtering, we're going to develop an algorithm that learns to match users and movies. Previously, we were predicting the rating of user j on movie i as $w^{(j)} \cdot x^{(i)} + b^{(j)}$. In order to develop content-based filtering, I'm going to get rid of $b^{(j)}$. It turns out this won't hurt the performance of the content-based filtering at all. Instead of writing $w^{(j)}$ for a user j and $x^{(i)}$ for a movie i, I'm instead going to just replace this notation with $v_u^{(j)}$. This v here stands for a vector. There'll be a list of numbers computed for user j and the u subscript here stands for user.

Instead of $x^{(i)}$, I'm going to compute a separate vector $v_m^{(i)}$, to stand for the movie and for movie is what a superscript stands for. $v_u^{(j)}$ as a vector as a list of numbers computed from the features of user j and $v_m^{(i)}$ is a list of numbers computed from the features like the ones you saw on the previous slide of movie i.

If we're able to come up with an appropriate choice of these vectors, $v_u^{(j)}$ and $v_m^{(i)}$, then hopefully the dot product between these two vectors will be a good prediction of the rating that user j gives movie i. Just illustrate what a learning algorithm could come up with. If v, u, that is a user vector, turns out to capture the user's preferences, say is 4.9, 0.1, and so on.

Lists of numbers like that.

- The first number captures how much do they like romance movies.
- Then the second number captures how much do they like action movies and so on. Then v_m, the movie vector is 4.5, 0.2, and so on and so forth of

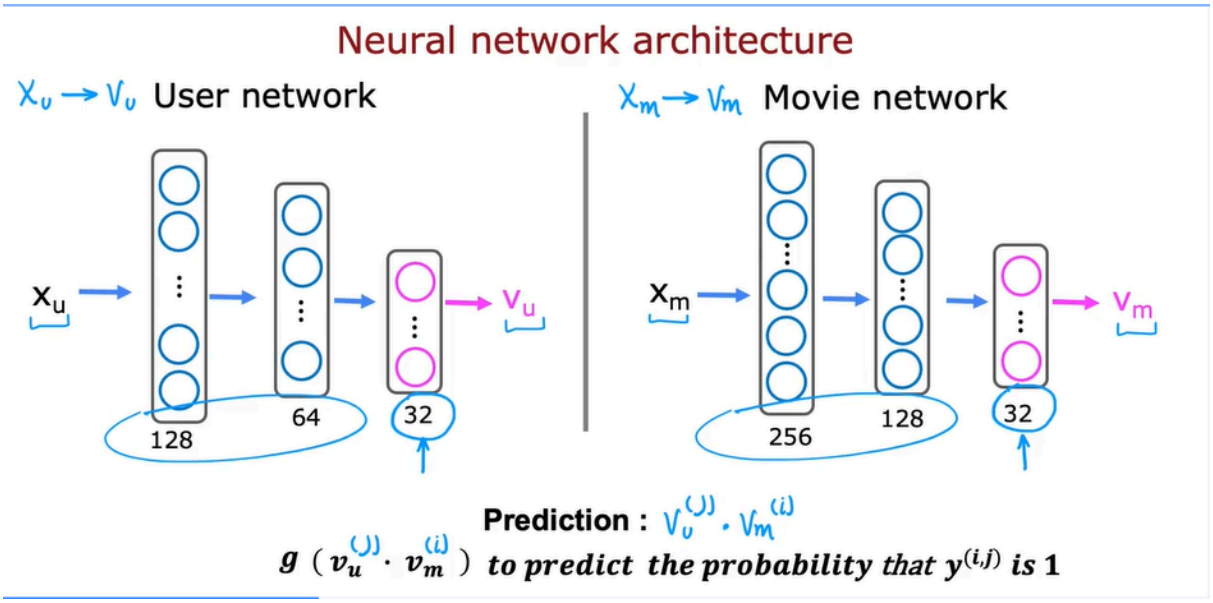
these numbers capturing how much is this a romance movie, how much is this an action movie, and so on.

Then the dot product, which multiplies these lists of numbers element-wise and then takes a sum, hopefully, will give a sense of how much this particular user will like this particular movie. The challenges given features of a user, say $x_u^{(j)}$, how can we compute this vector $v_u^{(j)}$ that represents succinctly or compactly the user's preferences? Similarly given features of a movie, how can we compute $v_m^{(i)}$? Notice that whereas $x_u^{(j)}$ and $x_m^{(i)}$ could be different in size, one could be very long lists of numbers, one could be much shorter list, v here ($v_u^{(j)}$ and $v_m^{(i)}$) have to be the same size.

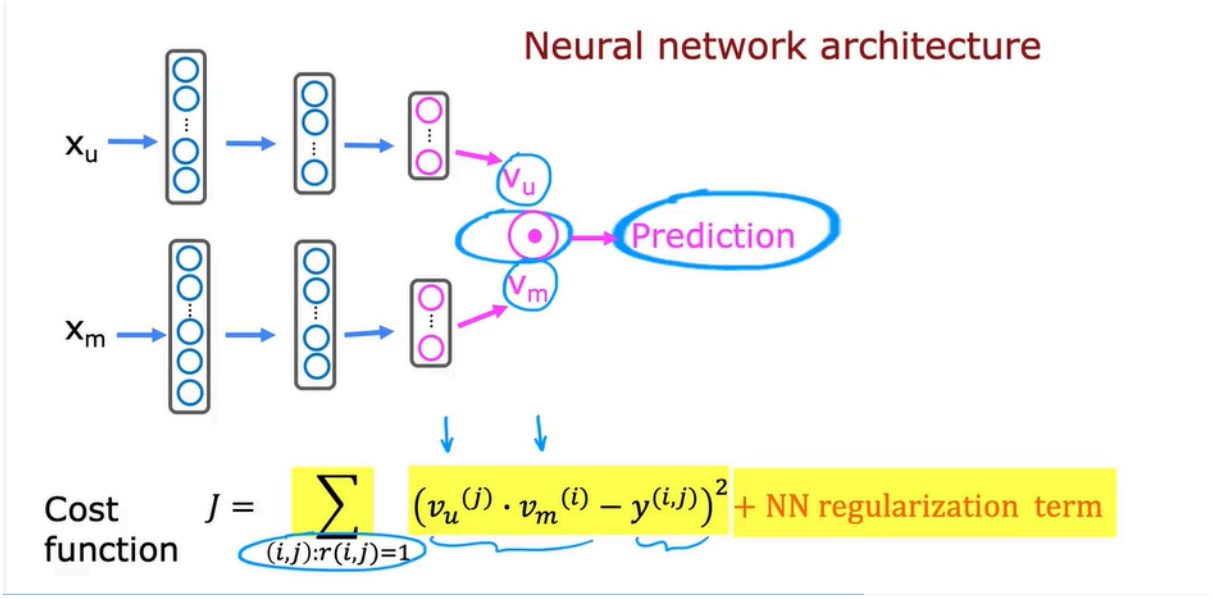
Because if you want to take a dot product between ($v_u^{(j)}$ and $v_m^{(i)}$), then both of them have to have the same dimensions such as maybe both of these are say 32 numbers.

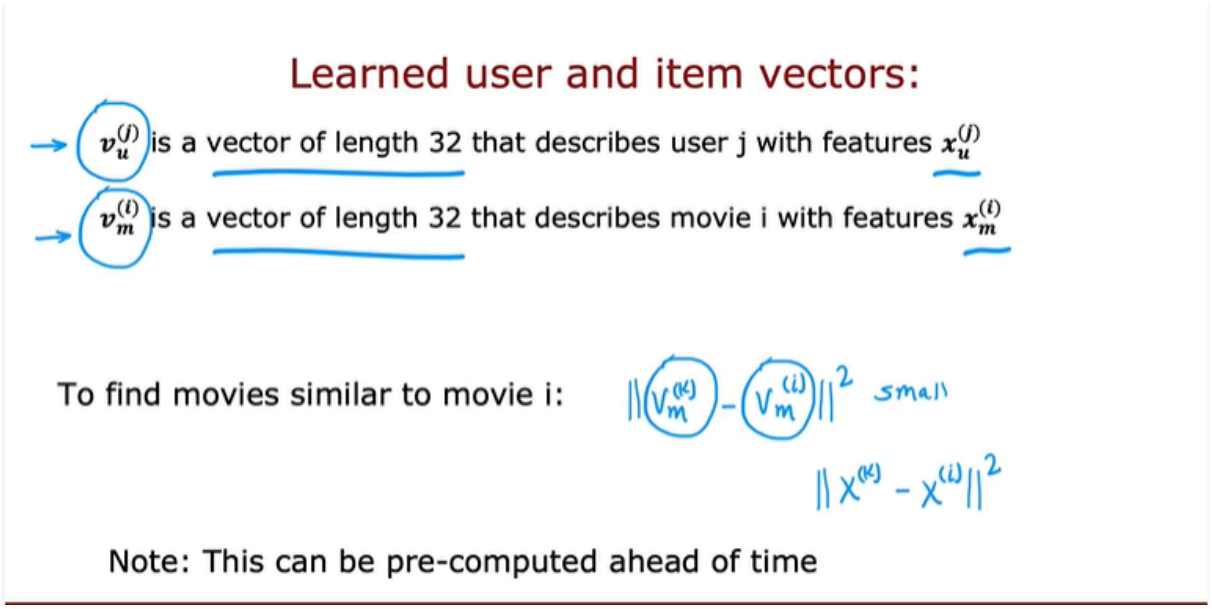
To summarize, in collaborative filtering, we had number of users give ratings of different items. In contrast, in content-based filtering, we have features of users and features of items and we want to find a way to find good matches between the users and the items. The way we're going to do so is to compute these vectors, $v_u^{(j)}$ for the users and $v_m^{(i)}$ for the items over the movies, and then take dot products between them to try to find good matches.

Deep learning for content-based filtering



Notice that the user network and the movie network can hypothetically have different numbers of hidden layers and different numbers of units per hidden layer. All the output layer needs to have the same size of the same dimension. In the description you've seen so far, we were predicting the 1-5 or 0-5 star movie rating. If we had binary labels, if y was to the user like or favor an item, then you can also modify this algorithm to output.





Given a specific movie, what if you want to find other movies similar to it? Well, this vector $v_m^{(i)}$ describes the movie i.

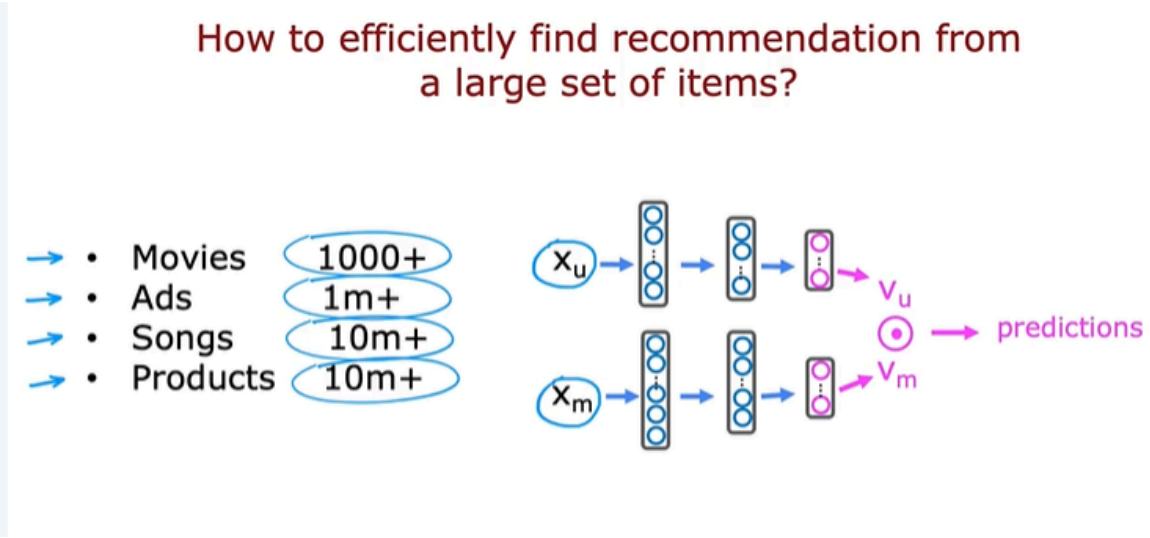
If you want to find other movies similar to it, you can then look for other movies k so that the distance between the vector describing movie k and the vector describing movie i, that the squared distance is small. This expression plays a role similar to what we had previously with collaborative filtering, where we talked about finding a movie with features $x^{(k)}$ that was similar to the features $x^{(i)}$.

Thus, with this approach, you can also find items similar to a given item. One final note, this can be pre-computed ahead of time. By that I mean, you can run a compute server overnight to go through the list of all your movies and for every movie, find similar movies to it, so that tomorrow, if a user comes to the website and they're browsing a specific movie, you can already have pre-computed to 10 or 20 most similar movies to show to the user at that time.

The fact that you can pre-compute ahead of time what's similar to a given movie, will turn out to be important later when we talk about scaling up this approach to a very large catalog of movies. That's how you can use deep learning to build a content-based filtering algorithm.

One notes, if you're implementing these algorithms in practice, I find that developers often end up spending a lot of time carefully designing the features needed to feed into these content-based filtering algorithms. If we end up building one of these systems commercially, it may be worth spending some time engineering good features for this application as well. In terms of these applications, one limitation that the algorithm as we've described it is it can be computationally very expensive to run if you have a large catalog of a lot of different movies you may want to recommend.

Recommending from a large catalogue

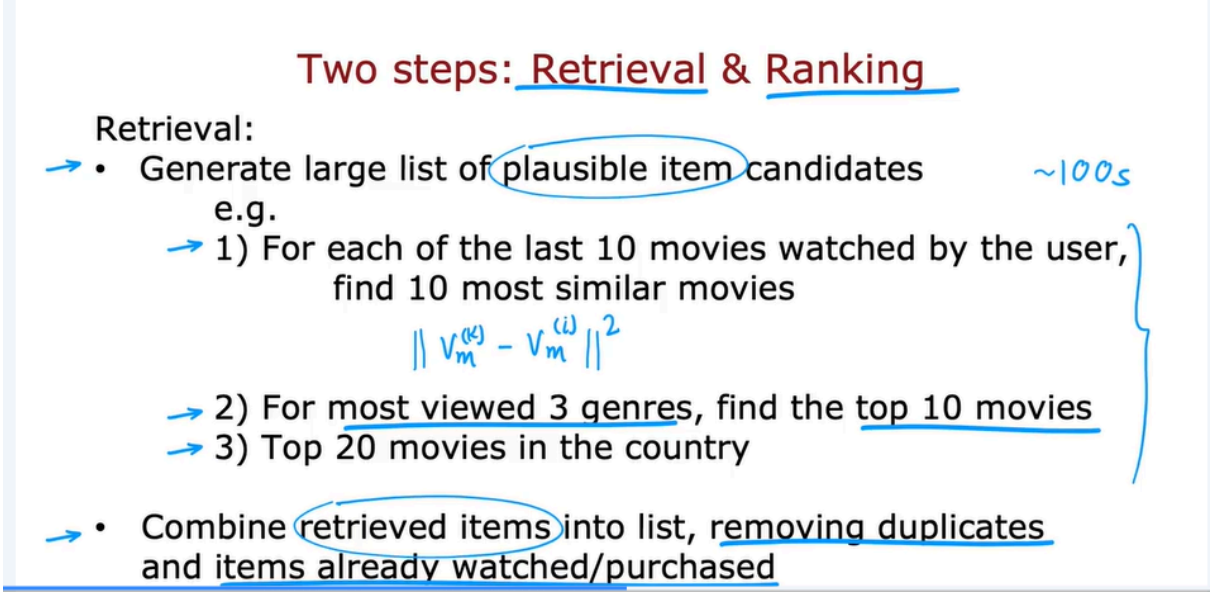


From a catalog of thousands or millions or 10s of millions or even more items. How do you do this efficiently computationally, let's take a look.

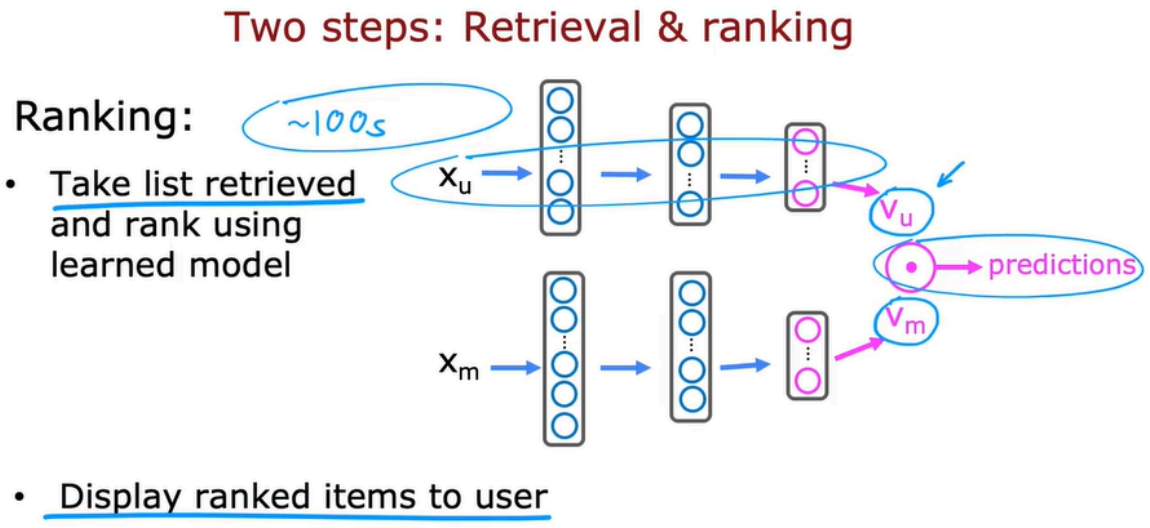
Here's in your network we've been using to make predictions about how a user might rate an item. Today a large movie streaming site may have thousands of movies or a system that is trying to decide what ad to show. May have a catalog of millions of ads to choose from. Or a music streaming sites may have 10s of millions of songs to choose from. And large online shopping sites can have millions or even 10s of millions of products to choose from. When a user shows up on your website, they have some feature Xu. But if you need to take thousands of millions of items to feed through this neural network in order to compute in the product.

To figure out which products you should recommend, then having to run neural network inference. Thousands of millions of times every time a user shows up on your website becomes computationally infeasible.

Retrieval



Ranking

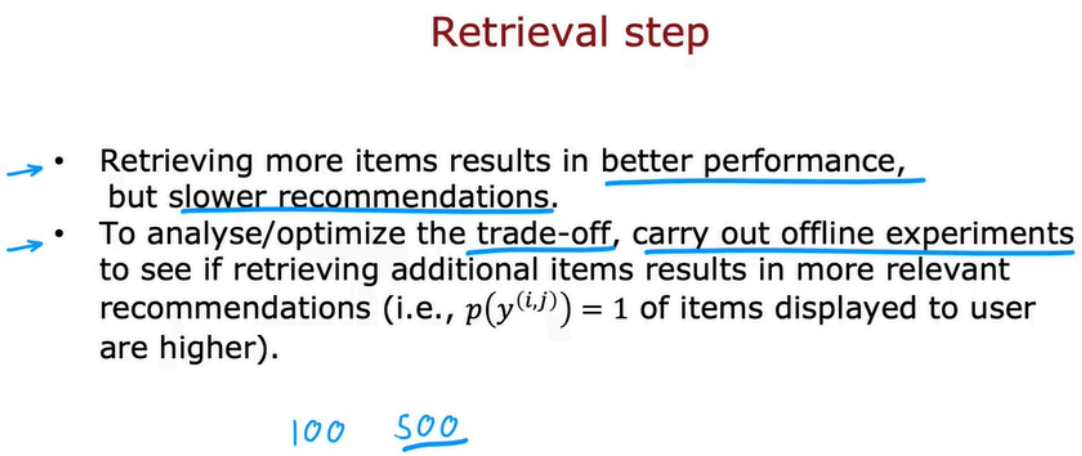


During the ranking step you will take the list retrieved during the retrieval step. So this may be just hundreds of possible movies and rank them using the learned model.

And what that means is you will feed the user feature vector and the movie feature actor into this neural network. And for each of the user movie pairs compute the predicted rating. And based on this, you now have all of the say 100+ movies, the ones that the user is most likely to give a high rating to. And then you can just display the rank list of items to the user depending on what you think the user will give.

The highest rating to one additional optimization is that if you have computed v_m. For all the movies in advance, then all you need to do is to do inference on this part of the neural network a single time to compute v_u. And then take that v_u they just computed for the user on your website right now. And take the inner product between v_u and v_m.

For the movies that you have retrieved during the retrieval step. So this computation can be done relatively quickly. If the retrieval step just brings up say 100s of movies, one of the decisions you need to make for this algorithm is how many items do you want to retrieve during the retrieval step? To feed into the more accurate ranking step.



During the retrieval step, retrieving more items will tend to result in better performance. But the algorithm will end up being slower to analyze or to optimize the trade off between how many items to retrieve to retrieve 100 or 500 or 1000 items.

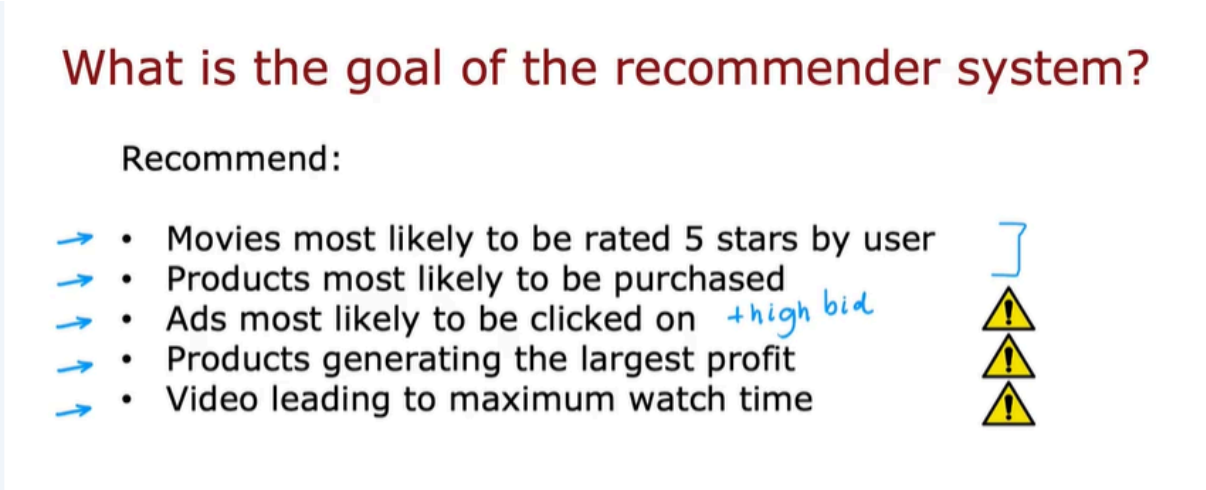
I would recommend carrying out offline experiments to see how much retrieving additional items results in more relevant recommendations. And in particular, if the estimated probability that $y^{(i,j)}$. Is equal to 1 according to your neural network model. Or if the estimated rating of Y being high of the retrieve items according to your model's prediction ends up being much higher. If only you were to retrieve say 500 items instead of only 100 items, then that would argue for maybe retrieving more items.

Even if it slows down the algorithm a bit. But with the separate retrieval step and the ranking step, this allows many recommender systems today to give

both fast as well as accurate results. Because the retrieval step tries to prune out a lot of items that are just not worth doing the more detailed influence and inner product on. And then the ranking step makes a more careful prediction for what are the items that the user is actually likely to enjoy so that's it.

This is how you make your recommender system work efficiently even on very large catalogs of movies or products or what have you. Now, it turns out that as commercially important as our recommender systems, there are some significant ethical issues associated with them as well. And unfortunately there have been recommender systems that have created harm. So as you build your own recommender system, I hope you take an ethical approach and use it to serve your users. And society as large as well as yourself and the company that you might be working for.

Ethical use of recommender systems



Even though recommender systems have been very profitable for some businesses, that happens, some use cases that have left people and society at large worse off. However, you use recommender systems or for that matter other learning algorithms, I hope you only do things that make society at large and people better off.

Let's take a look at some of the problematic use cases of recommender systems, as well as ameliorations to reduce harm or to increase the amount of good that they can do. As you've seen in the last few videos, there are many ways of configuring a recommender system. When we saw binary labels, the label y could be, does a user engage or did they click or did they explicitly like an item?

When designing a recommender system, choices in setting the goal of the recommender system and a lot of choices and deciding what to recommend to users.

For example, you can decide to recommend to users movies most likely to be rated five stars by that user. That seems fine. That seems like a fine way to show users movies that they would like.

Or maybe you can recommend to the user products that they are most likely to purchase. That seems like a very reasonable use of a recommender system as well.

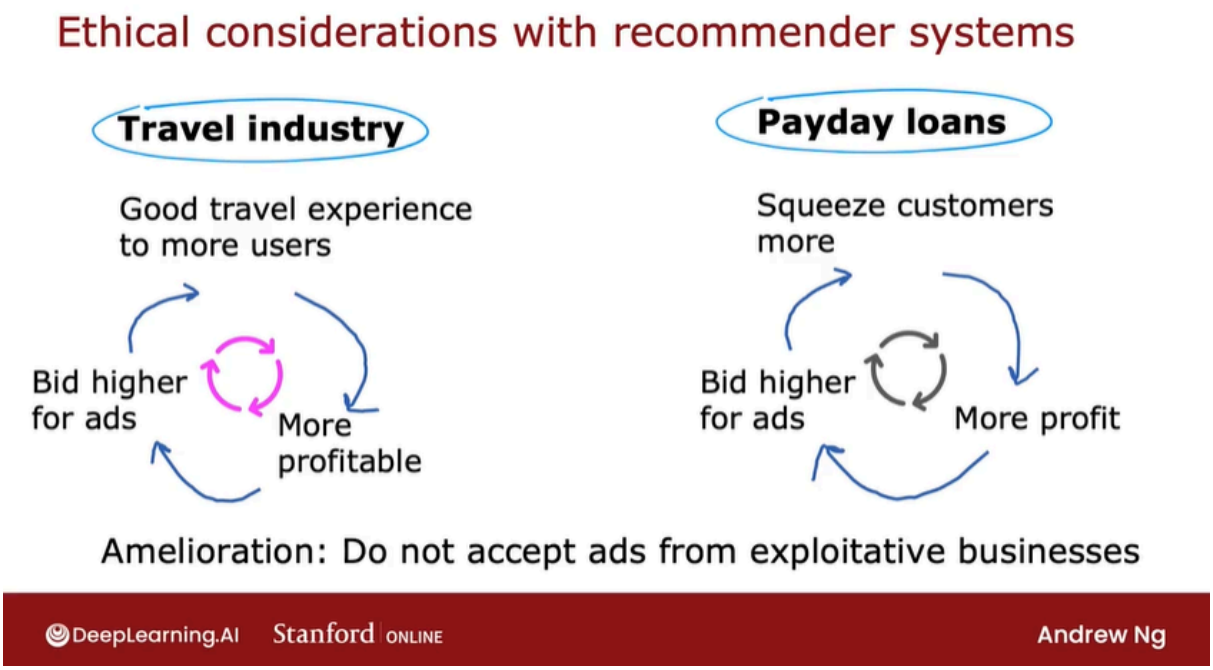
Versions of recommender systems can also be used to decide what ads to show to a user. One thing you could do is to recommend or really to show to the user ads that are most likely to be clicked on.

- Actually, what many companies will do is try to show ads that are likely to be clicked on and where the advertiser had put in a high bid because for many ad models, the revenue that the company collects depends on whether the ad was clicked on and what the advertiser had bid per-click. While this is a profit-maximizing strategy, there are also some possible negative implications of this type of advertising. I'll give a specific example on the next slide.
- One other thing that many companies do is try to recommend products that generate the largest profit. If you go to a website and search for a product today, there are many websites that are not showing you the most relevant product or the product that you are most likely to purchase. But is instead trying to show you the products that will generate the largest profit for the company. If a certain product is more profitable for them, because they can buy it more cheaply and sell it at a higher price, that gets ranked higher in the recommendations. Now, many companies view a pressure to maximize profit.

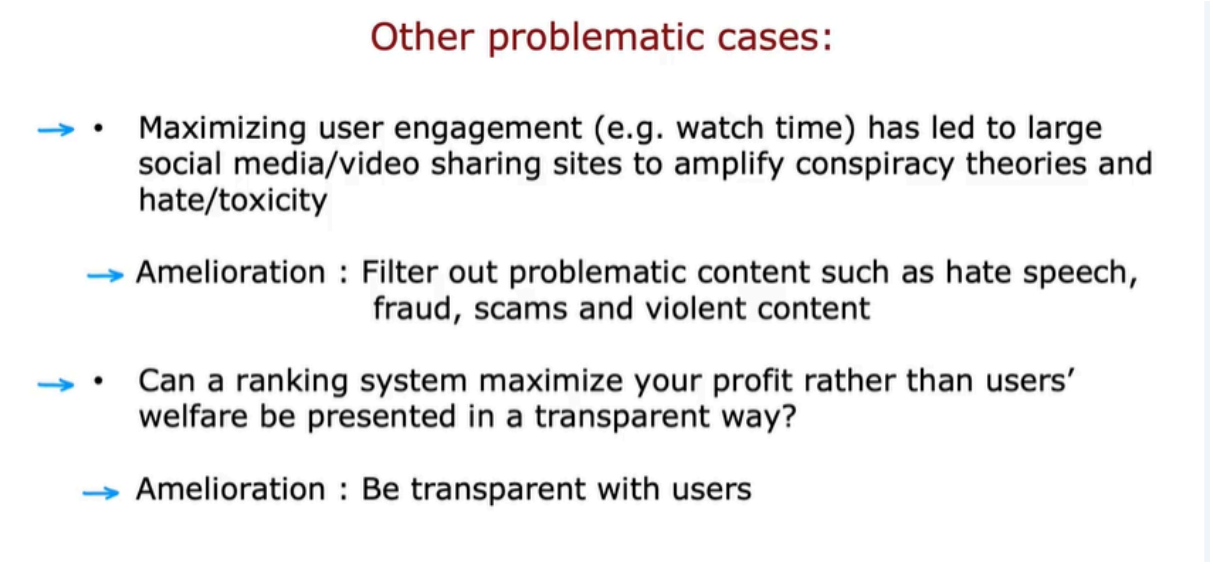
This doesn't seem like an unreasonable thing to do but on the flip side, from the user perspective, when a website recommends to you a product, sometimes it feels it could be nice if the website was transparent with you about the criteria by which it is deciding what to show you. Is it trying to maximize their profits or trying to show you things that are most useful to you?

On video websites or social media websites, a recommender system can also be modified to try to show you the content that leads to the maximum watch time. Specifically, websites that are an ad revenue tend to have an incentive to

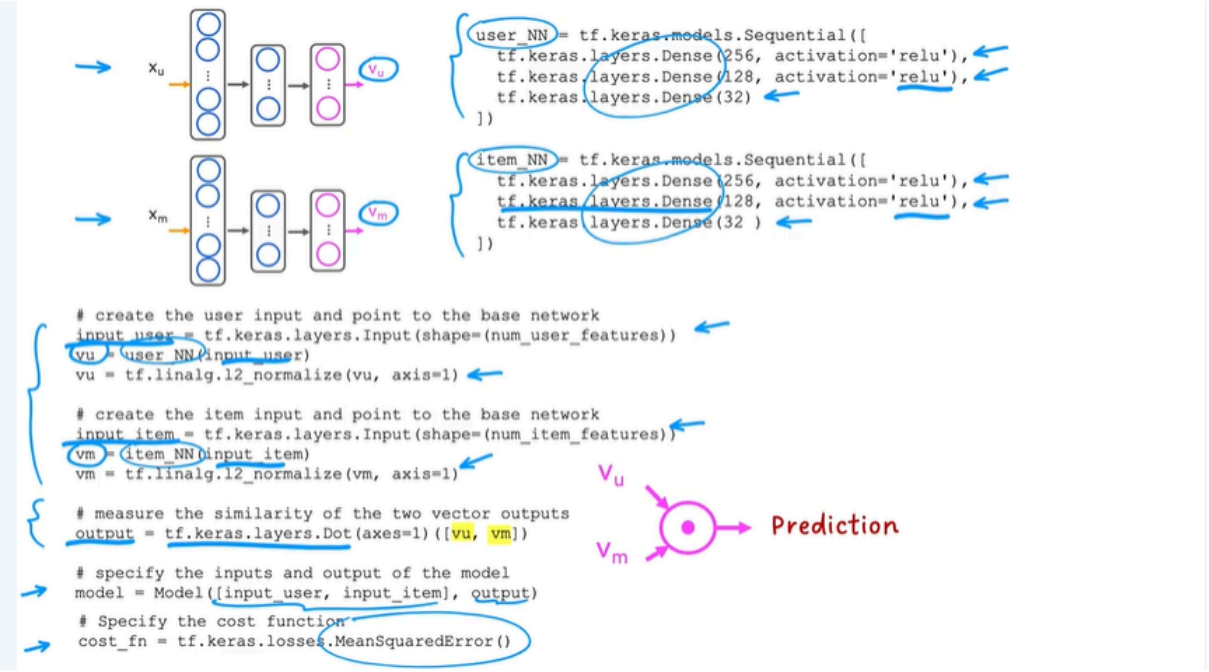
keep you on the website for a long time. Trying to maximize the time you spend on the site is one way for the site to try to get more of your time so they can show you more ads. Recommender systems today are used to try to maximize user engagement or to maximize the amount of time that someone spends on a site or a specific app. Whereas the first two of these seem quite innocuous, the third, fourth, and fifth, they may be just fine. They may not cause any harm at all. Or they could also be problematic use cases for recommender systems. Let's take a deeper look at some of these potentially problematic use cases.



Amelioration: Sự cải thiện



TensorFlow implementation of content-based filtering



It turns out that there's one other step that I didn't talk about previously, but if you do this, which is normalize the length of the vector vu, that makes the algorithm work a bit better. TensorFlows has this l2 normalized motion that normalizes the vector, is also called normalizing the l2 norm of the vector, hence the name of the function

4. You have built a recommendation system to retrieve musical pieces from a large database of music, and have an algorithm that uses separate retrieval and ranking steps. If you modify the algorithm to add more musical pieces to the retrieved list (i.e., the retrieval step returns more items), which of these are likely to happen? Check all that apply.

1 / 1 point

- [x] The quality of recommendations made to users should stay the same or improve.

  A larger retrieval list gives the ranking system more options to choose from which should maintain or improve recommendations.

- [x] The system's response time might increase (i.e., users have to wait longer to get recommendations)

  A larger retrieval list may take longer to process which may *increase* response time.

- [ ] The system's response time might decrease (i.e., users get recommendations more quickly)
- [ ] The quality of recommendations made to users should stay the same or worsen.